

CloneCalls

Security Requirements, Risk Management & Threat Modeling

Deliverable 1

1. Project Overview

CloneCalls is a web application that allows users to create an AI-powered voice clone that answers calls on their behalf. When a user is unavailable, they brief their clone with daily context — their schedule, current status, and messages to relay. An authorised caller can then query the clone via text or voice and receive a spoken, AI-generated response in real time.

The application is built on a Python FastAPI backend, integrated with Supabase for authentication, Pinecone for vector memory storage, OpenAI GPT-4o for natural language generation and audio transcription, and ElevenLabs for voice synthesis. A single-page HTML and JavaScript frontend provides the user interface.

1.1 System Components

Component	Technology	Description
Backend API	Python FastAPI	REST API handling all business logic, authentication, and data routing
Authentication	Supabase Auth	JWT issuance, password hashing, and user account management
Vector Memory	Pinecone DB	Stores and retrieves user briefing embeddings per user, per day
Language Model	OpenAI GPT-4o-mini	Generates AI clone responses; Whisper transcribes uploaded voice audio
Voice Synthesis	ElevenLabs TTS	Converts AI text responses into spoken audio for the caller
Frontend	HTML / Vanilla JS	Single-page application served via Live Server on localhost

1.2 Security Context

CloneCalls handles three categories of sensitive data that make security a primary concern rather than an afterthought:

- Personal context data — daily schedules, location, availability, and private messages stored in a third-party vector database
- Authentication credentials — email addresses and passwords used to access the system
- Voice recordings — raw audio files uploaded by callers and processed by external AI APIs

Each category presents distinct security risks that must be addressed at the design stage. This document defines the security requirements, risk register, and threat model that will guide the secure implementation in subsequent deliverables.

2. GitHub Repository

The project repository has been created and is available at the following address:

Repository URL

<https://github.com/Yime1705/CloneCalling>

All source code, documentation, and deliverable reports are submitted through this repository. Each deliverable is maintained in a separate folder (Deliverable-1, Deliverable-2, Deliverable-3) as required by the course submission guidelines.

2.1 Repository Structure

Folder / File	Contents
Deliverable-1/	This report and any supporting planning documentation
Deliverable-2/	Initial implementation source code and secure coding report
Deliverable-3/	Security testing results, code review, and final report
backend/	FastAPI Python application (main.py, routes.py, security.py, schemas.py, config.py)
index.html	Single-page frontend application
.gitignore	Excludes .env, __pycache__, and other sensitive or generated files
.env (excluded)	API keys and configuration — never committed to the repository

3. Security Requirements and Planning

Security requirements for CloneCalls were derived from the sensitivity of the data handled, the multi-user nature of the system, and the involvement of multiple third-party APIs. Requirements are classified by category and mapped to implementation controls.

3.1 Functional Security Requirements

Authentication Requirements

ID	Requirement	Priority	Implementation Approach
SR-01	Users must authenticate before accessing any protected functionality	Critical	JWT Bearer token via Supabase on all protected endpoints
SR-02	Passwords must be at least 8 characters in length	Critical	Pydantic schema validation (min_length=8) in schemas.py
SR-03	Passwords must be stored as cryptographic hashes — never plaintext	Critical	Supabase Auth handles all hashing; passwords never stored or processed in application code
SR-04	Session tokens must expire and must be validated on every request	Critical	Supabase JWT expiry; server-side validation via <code>supabase.auth.get_user()</code> on each call
SR-05	Login failures must not reveal which field (email or password) is incorrect	High	Generic 'Invalid email or password' error message on all login failures

Authorisation Requirements

ID	Requirement	Priority	Implementation Approach
SR-06	A user must only be able to access briefings they are explicitly authorised to access	Critical	Triple-filter Pinecone query: owner + allowed_caller + date must all match

SR-07	User identity used in data queries must come from the server-validated token, not from request body fields	Critical	current_email always extracted from the Supabase token in get_current_user()
SR-08	Briefing access must be scoped to the current day only — historical briefings must be inaccessible	High	'date' filter included in all Pinecone queries using today's date
SR-09	A user must name exactly one authorised caller per briefing upload	High	allowed_caller EmailStr field required in DailyBriefing Pydantic model

Data Protection Requirements

ID	Requirement	Priority	Implementation Approach
SR-10	Personally Identifiable Information must be stripped from briefing text before storage	Critical	Regex scrubbing of credit cards, SSNs, and phone numbers in scrub_pii()
SR-11	API keys and secrets must never appear in source code or version control	Critical	.env file for all credentials; .gitignore excludes .env from commits
SR-12	The application must not expose internal error details or stack traces to clients	High	Generic error messages; raw exceptions caught and replaced before returning responses
SR-13	Audio recordings must not persist on the server filesystem after processing	High	Temporary file deletion in finally block — guaranteed even if transcription fails
SR-14	API documentation endpoints must be disabled in production	Medium	docs_url=None, redoc_url=None, openapi_url=None in FastAPI constructor

Input Validation Requirements

ID	Requirement	Priority	Implementation Approach
SR-15	All API request bodies must be validated for type, format, and length before processing	Critical	Pydantic models in schemas.py with EmailStr, Field(min_length, max_length)
SR-16	Audio file uploads must be capped at a maximum size before reading into memory	Critical	10 MB cap enforced before memory allocation; returns HTTP 413 on excess
SR-17	Email addresses in query parameters must be validated against a regex pattern	High	Query(pattern=r'^[^\s]+@[^\s]+\.[^\s]+\$') on target_email params
SR-18	The AI system prompt must constrain the model to only use data from the briefing context	High	Explicit CRITICAL SECURITY RULE in system prompt; context scoped to top_k=3 filtered results

Network and Infrastructure Requirements

ID	Requirement	Priority	Implementation Approach
SR-19	Cross-Origin Resource Sharing must be restricted to explicitly allowed origins	High	CORS allowlist loaded from .env; no wildcard (*) used
SR-20	Security headers must be set on all API responses	High	Middleware adds X-Frame-Options, X-Content-Type-Options, CSP, Cache-Control, Referrer-Policy

SR-21	All outbound calls to external APIs must include a timeout to prevent hung connections	Medium	timeout=30 on all ElevenLabs requests.post() calls
SR-22	External API keys must not appear in URL query parameters	Medium	All keys passed in HTTP headers; never appended to request URLs

3.2 Non-Functional Security Requirements

ID	Requirement	Rationale
NFR-01	All secrets must fail fast at startup — the server must not start with incomplete configuration	Prevents silent partial-configuration failures that could expose the system in an insecure state
NFR-02	Error messages must be generic enough to not aid an attacker but specific enough to be actionable for a legitimate user	Balances usability and security — avoids information leakage while remaining useful
NFR-03	The codebase must never contain hardcoded credentials, even in comments	Prevents accidental credential exposure in version control history
NFR-04	Security controls must be enforced server-side and must not rely on frontend validation alone	Frontend validation can be bypassed by any HTTP client — backend is the authoritative security boundary

3.3 Security Constraints and Acknowledged Limitations

The following security controls are required in a production deployment but cannot be implemented in the current localhost development environment. They are acknowledged here and will be documented in full in the final deliverable.

Control	Constraint
Multi-Factor Authentication	Requires a registered domain for Supabase OTP redirect URLs

HTTPS / TLS Encryption	SSL certificates require a registered domain name
HttpOnly Secure Cookies	The Secure flag is ignored by browsers over HTTP
Email Verification on Signup	Supabase verification links need a real domain as the callback URL
OAuth / Single Sign-On	OAuth providers reject 127.0.0.1 as a valid redirect URI
Production Rate Limiting	Meaningful only at the reverse proxy or CDN level with persistent state
WebAuthn / Passkeys	Browser enforces that the rpId must be a registered domain

4. Risk Management

Risk assessment uses a 3x3 likelihood-impact matrix. Each risk is rated on likelihood (Low / Medium / High) and impact (Low / Medium / High). The overall risk level is the product of these two dimensions.

4.1 Risk Rating Matrix

		Impact: Low	Impact: Medium	Impact: High	
Likelihood	High	Medium	High	Critical	
	Medium	Low	Medium	High	
	Low	Low	Low	Medium	

4.2 Risk Register

#	Threat	Likelihood	Impact	Risk	Mitigation
---	--------	------------	--------	------	------------

R-01	Unauthorised access to another user's briefing memories	Medium	High	High	Triple-filter Pinecone query (owner + allowed_caller + date); server-side identity enforcement
R-02	JWT token forged or stolen to impersonate a user	Low	High	Medium	Server-side Supabase token validation on every request; tokens never decoded locally
R-03	PII (SSN, credit card, phone) stored in plaintext in vector database	High	High	High	Regex scrubbing of all briefing text before storage; raw text never reaches Pinecone
R-04	Prompt injection — caller manipulates AI clone to reveal out-of-scope data	Medium	High	High	Constrained system prompt with hard security rule; context scoped to top_k=3 filtered results
R-05	Denial of service via oversized audio file upload	Medium	Medium	Medium	10 MB file size cap enforced before memory read; returns HTTP 413 immediately
R-06	API keys committed to GitHub repository accidentally	Low	High	Medium	All keys in .env; .gitignore excludes .env; fail-fast startup validation

R-07	Credential brute force attack on /login endpoint	Medium	High	High	Generic error messages; Supabase built-in account lockout after repeated failures
R-08	Clickjacking — application embedded in malicious iframe	Low	Medium	Low	X-Frame-Options: DENY; CSP frame-ancestors 'none' on all API responses
R-09	XSS via API error messages rendered in the browser	Medium	Medium	Medium	showMsg() uses textContent; showMsgHTML() used only for hardcoded internal HTML
R-10	Email enumeration via login endpoint distinguishing email vs password error	Medium	Medium	Medium	Single generic error message: 'Invalid email or password' for all login failures
R-11	API endpoint enumeration via Swagger UI	Low	Medium	Low	docs_url=None, redoc_url=None, openapi_url=None in FastAPI constructor
R-12	Sensitive audio recordings persisting on the server filesystem	Low	High	Medium	Temporary file deletion in finally block — guaranteed even if transcription fails
R-13	Hung external API connection stalling a server thread indefinitely	Medium	Medium	Medium	30-second timeout on all ElevenLabs outbound requests

R-14	MIME-type sniffing causing browser to misinterpret API response	Low	Low	Low	X-Content-Type-Options: nosniff header on all API responses
R-15	Sensitive API responses cached in browser or proxy	Low	Medium	Low	Cache-Control: no-store on all API responses via middleware
R-16	Cross-site request forgery from an unauthorised origin	Low	Medium	Low	CORS allowlist from .env; no wildcard; credentials required; restricted methods
R-17	Data interception on network (tokens, briefings in transit)	Low	High	Medium	HTTPS planned for production deployment; acknowledged localhost limitation
R-18	Missing required API key causes silent partial-configuration failure	Low	Medium	Low	Fail-fast startup validation raises RuntimeError if any key is absent

4.3 Risk Priority Summary

Risk Level	Count	Risks
High	5	R-01 (unauthorised data access), R-03 (PII storage), R-04 (prompt injection), R-07 (brute force), R-04 (prompt injection)
Medium	8	R-02, R-05, R-06, R-09, R-10, R-12, R-13, R-17
Low	5	R-08, R-11, R-14, R-15, R-16, R-18

All five High-risk items are addressed by security controls implemented in the initial codebase. The controls are documented in full in Deliverable 2.

5. Threat Modeling

5.1 System Architecture Overview

The CloneCalls architecture spans three tiers: a browser-based frontend, a Python FastAPI backend, and a set of external cloud services. The diagram below describes the data flow and trust boundaries between components.

Layer	Components	Trust Level
Frontend (Browser)	index.html — Auth UI, Clone UI, Call UI, Status UI	Untrusted — all input must be validated server-side
API Boundary	FastAPI — CORS middleware, Security headers, JWT validation	Partially trusted — enforces authentication and input validation
Application Logic	routes.py — Business logic, Pinecone queries, AI calls	Trusted — runs server-side with validated identity
Authentication	Supabase Auth — JWT issuance and validation	Trusted external — industry-standard managed service
Vector Store	Pinecone — Per-user briefing embeddings	Trusted external — accessed only with validated server credentials
AI Services	OpenAI GPT-4o, ElevenLabs TTS	Trusted external — accessed with server-held API keys over HTTPS

5.2 Data Flow Description

User Registration and Login

1. User submits email and password via the frontend signup/login form
2. Frontend sends credentials to POST /signup or POST /login on the FastAPI backend
3. Backend validates input format and length via Pydantic before touching Supabase

4. Supabase creates the account and hashes the password — the application never handles plaintext passwords
5. On login, Supabase issues a JWT; the backend returns it to the frontend
6. Frontend stores the JWT in localStorage and attaches it as Authorization: Bearer on all subsequent requests

Briefing Upload

7. Authenticated user writes a briefing and specifies one allowed_caller email
8. Frontend POSTs to /upload_briefing with JWT in the Authorization header
9. Backend validates the JWT via Supabase; extracts owner email from the token
10. scrub_pii() strips credit card numbers, SSNs, and phone numbers from the briefing text
11. Cleaned text is embedded via OpenAI and stored in Pinecone with three metadata fields: owner, allowed_caller, date

Voice or Text Call

12. Caller provides the target user's email and their own JWT
13. Backend validates JWT; extracts caller email from the token
14. Pinecone is queried with a triple filter: owner=target, allowed_caller=caller, date=today
15. If audio uploaded, it is written to a temporary file, transcribed via Whisper, then immediately deleted in a finally block
16. Retrieved memories are injected into a constrained AI system prompt
17. GPT-4o-mini generates the response; ElevenLabs converts it to audio; audio is returned to caller

5.3 Trust Boundaries

Boundary	Description	Security Control
Browser → API	All data from the browser is untrusted	Pydantic validation, JWT authentication on every request
API → Supabase	Token validation crosses this boundary on every request	Server-side supabase.auth.get_user() — never local decode
API → Pinecone	Only the server holds the Pinecone API key	Triple-filter query enforced server-side; key in .env
API → OpenAI	Briefing text and audio cross this boundary	PII scrubbing before text; audio deleted after transcription

API → ElevenLabs	AI response text crosses this boundary	API key in headers; 30-second timeout; 502 on failure
User A → User B data	Callers must not access other users' memories	Triple-filter enforces owner, caller, and date simultaneously

5.4 STRIDE Threat Analysis

STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) is applied to each trust boundary and major component.

STRIDE Category	Threat	Attack Vector	Security Control
Spoofing	Attacker forges a JWT to impersonate a registered user	Crafted token in Authorization header	Server-side Supabase validation on every request — <code>supabase.auth.get_user()</code>
Spoofing	Caller claims to be an authorised <code>allowed_caller</code>	Modifies request body or query parameters	<code>current_email</code> always from validated token, not from any user input
Tampering	User modifies <code>target_email</code> to access another user's clone	Changes query parameter value	Triple-filter query — all three fields must match simultaneously
Tampering	Caller injects prompt commands to manipulate the AI	Submits adversarial text as the call message	Constrained system prompt with explicit CRITICAL SECURITY RULE
Tampering	Attacker embeds the app in an iframe to capture clicks	Iframe with identical-looking overlay	X-Frame-Options: DENY; CSP frame-ancestors 'none'

Repudiation	User denies uploading a specific briefing	Claims no record of the action	Owner email from token stored as metadata on every Pinecone record
Information Disclosure	PII included in briefing text stored in third-party DB	User types SSN or credit card in briefing field	Regex PII scrubbing applied before any embedding or storage
Information Disclosure	Attacker enumerates API endpoints via Swagger UI	Navigates to /docs or /openapi.json	All documentation endpoints disabled by default
Information Disclosure	Login endpoint reveals whether email exists	Tests 'wrong email' vs 'wrong password' responses	Single generic error message regardless of which field failed
Information Disclosure	API keys appear in server logs via URL parameters	Keys appended to request URLs	All API keys passed in HTTP headers only
Denial of Service	Attacker uploads enormous audio file to exhaust memory	Oversized multipart upload to /voice_call	10 MB cap enforced before memory read; HTTP 413 returned
Denial of Service	Slow ElevenLabs response stalls server thread indefinitely	ElevenLabs API becomes unresponsive	30-second timeout on all outbound ElevenLabs calls
Elev. of Privilege	Attacker brute forces weak passwords on /login	Automated credential stuffing	Generic errors; Supabase account lockout; password min length 8

Elev. of Privilege	Attacker accesses another user's memories via IDOR	Changes owner field in query parameters	Server ignores client-supplied identity; uses only token-derived email
--------------------	--	---	--

5.5 Security Controls Summary

The following table summarises all planned security controls, the requirement they satisfy, and where they will be implemented in the codebase.

#	Security Control	Satisfies	File
1	JWT Bearer token authentication via Supabase	SR-01, SR-04	security.py
2	Server-side token validation on every request	SR-04, SR-07	security.py
3	Password minimum 8 characters (Pydantic)	SR-02	schemas.py
4	Supabase-managed password hashing	SR-03	Supabase Auth
5	Triple-filter Pinecone queries (owner + caller + date)	SR-06, SR-08	routes.py
6	Server-side identity enforcement from token	SR-07	routes.py
7	Pydantic schema validation on all request bodies	SR-15	schemas.py
8	Query(pattern=...) on email query parameters	SR-17	routes.py
9	10 MB audio file size cap	SR-16	routes.py

10	Temporary audio file deletion in finally block	SR-13	routes.py
11	Regex PII scrubbing before Pinecone storage	SR-10	security.py
12	Constrained AI system prompt with hard security rule	SR-18	routes.py
13	CORS strict origin allowlist from .env	SR-19	main.py
14	Security response headers middleware (6 headers)	SR-20	main.py
15	All API keys in .env excluded from version control	SR-11	config.py
16	Fail-fast startup validation of all required keys	NFR-01	config.py
17	Generic error messages — no information leakage	SR-05, SR-12	routes.py
18	API documentation endpoints disabled	SR-14	main.py
19	30-second timeout on all ElevenLabs calls	SR-21	routes.py
20	API keys in HTTP headers not URL parameters	SR-22	routes.py
21	Frontend: showMsg() uses textContent not innerHTML	SR-12	index.html
22	Frontend: JWT stored and cleared correctly	SR-04	index.html

